

Q1)

2) $f(n) = (n^2 - 3n)^2$, $g(n) = 5n^3 + n$.

$$\lim_{n \rightarrow \infty} \frac{(n^2 - 3n)^2}{5n^3 + n} = \frac{\infty}{\infty} \xrightarrow{\text{L'Hopital}} \lim_{n \rightarrow \infty} \frac{2(n^2 - 3n)(2n - 3)}{15n^2 + 1} = \frac{\infty}{\infty}$$

$$\lim_{n \rightarrow \infty} \frac{4n^3 - 18n^2 + 18n}{15n^2 + 1} \xrightarrow{\text{L'Hopital}} \frac{12n^2 - 36n + 18}{30n} \rightarrow \lim_{n \rightarrow \infty} \frac{2n^2 - 6n + 3}{5n} = \frac{\infty}{\infty}$$

$$\lim_{n \rightarrow \infty} \frac{4n - 6}{5} = \infty \Rightarrow \text{Limit goes to } \infty, \text{ thus } f(n) = \Omega(g(n)).$$

b) $f(n) = n^3$, $g(n) = \log_2 n^4$.

$$\lim_{n \rightarrow \infty} \frac{n^3}{\ln n^4} = \frac{\infty}{\infty} \xrightarrow{\text{L'Hopital}} \lim_{n \rightarrow \infty} \frac{3n^2}{4n^3 \ln 2} = \frac{\infty}{\infty}$$

$$\log_a b^c = \frac{c b^{c-1}}{b^c \ln a}$$

$$\lim_{n \rightarrow \infty} \frac{3n^2}{4/n} \Rightarrow \lim_{n \rightarrow \infty} \frac{3n^2 \cdot n}{4} = \infty \Rightarrow \text{Limit goes to } \infty, \text{ thus } f(n) = \Omega(g(n)).$$

c) $f(n) = 5n \cdot \log_2(4n)$, $g(n) = n \cdot \log_2(5^n)$.

$$\lim_{n \rightarrow \infty} \frac{5n \ln(4n)}{n \ln(5^n)} = \frac{\infty}{\infty} \xrightarrow{\text{L'Hopital}} \left(5 \lim_{n \rightarrow \infty} \left(\frac{1}{\ln 2} \cdot \frac{5^n \ln 2}{5^n \ln 5} \right) \right)$$

$$\lim_{n \rightarrow \infty} \frac{5}{n \ln 5} = 0 \Rightarrow \text{Limit goes to } 0, \text{ thus } f(n) = o(g(n)).$$

d) $f(n) = n^n$, $g(n) = 10^n$

$$\lim_{n \rightarrow \infty} \frac{n^n}{10^n} = \frac{\infty}{\infty} \rightarrow \lim_{n \rightarrow \infty} \left(\frac{n}{10}\right)^n = \infty^\infty = \infty$$

↳ Limit goes to ∞ , thus $f(n) = \Omega(g(n))$.

e) $f(n) = 8n \cdot \sqrt[5]{2n}$, $g(n) = n \cdot \sqrt[3]{n}$

$$\lim_{n \rightarrow \infty} \frac{8n \cdot (2n)^{1/5}}{n \cdot n^{1/3}} = \frac{\infty}{\infty} \rightarrow \lim_{n \rightarrow \infty} \frac{8\sqrt[5]{2} \cdot n^{4/5}}{n^{4/3}} \rightarrow \lim_{n \rightarrow \infty} 8\sqrt[5]{2} \cdot n^{-3/15}$$

↳ $8\sqrt[5]{2} \lim_{n \rightarrow \infty} 1/n^{3/15} = 0 \rightarrow$ Limit goes to 0, thus $f(n) = O(g(n))$.

Q2)

```
a) static void methodA (String arr[]) {
    for (int i=0; i < arr.len; i++) // n : arr.length
        arr[i] = "";
}
```

↳ The funct. will loop for n times, so the time complexity is $O(n)$.

```
b) static void methodB (String arr[]) {
    for (i=0; i < arr.len; i++) // n : arr.length
        methodA (arr);
    for (j=0; j < arr.len; j++)
        print(arr[j]);
}
```

↳ First 'for' will execute for n times, & it will call method A. This creates a "nested loop" situation. Thus, first 'for' has the time complexity of $O(n^2)$.

↳ Second 'for' will also execute for n times. This makes its time complexity as $O(n)$.

$$O(n^2) + O(n) \Rightarrow O(n^2 + n) \rightarrow \text{drop lower level.} \\ \Rightarrow O(n^2)$$


```

c) static void methodC (arr[]) {
    for (i=0; i < arr.len; i++) // n: arr.length
        for (j=0; j < arr.len; j++)
            methodB(arr);
}

```

↳ Here we have a "nested loop". Thus, the time complexity is $O(n^2)$.

↳ But the inner loop calls method B which also has a time complexity of $O(n^2)$.

↳ Total is $O(n^2) \cdot O(n^2) \Rightarrow O(n^4)$.

```

d) static void methodD (arr[]) {
    for (i=0; i < arr.len; i++) {
        print(arr[i]);
        arr[i--] = "4";
    }
}

```

↳ In this function, the 'i' is always zero, because on "arr[i--] = '4';" line, it decreases i to -1, that makes i to always be smaller than zero.

↳ This creates an infinite loop, thus the function is **unsolvable** & has **no time complexity**.

```

e) static void methodE (arr[]) {
    for (i=0; i < arr.len; i++) // n: arr.length
        if (arr[i] == "0")
            break;
}

```

↳ Since we are analyzing worst-case scenario, the worst is that there is none space character in the array.

↳ Thus loop might execute for n times.

↳ if statements & breaks has constant time complexity $O(1)$.

↳ Total is $O(n)$.

Q3)

2) If the array is sorted in ascending order;

$arr = [1, 2, 3, 4, 5, 6, 7, \dots]$

→ Then the max-diff is directly:

```
$ VAR n as INTEGER
```

```
$ n = length(arr)
```

```
$ VAR max_diff as INTEGER
```

```
$ max_diff = arr[n-1] - arr[0]
```

→ Time complexity is constant time, $O(1)$.

b) If array is not sorted:

$arr = [2, 5, 3, 1, 8, 7, \dots]$

```
$ VAR n as INTEGER
```

```
$ n = length(arr)
```

```
$ VAR j as INTEGER
```

```
$ j = 0
```

```
$ VAR max_diff as INTEGER
```

```
$ max_diff = arr[1] - arr[0]
```

```
$ FOR i = 1 to n
```

```
$   IF arr[i] - arr[j] > max_diff THEN
```

```
$     max_diff = arr[i] - arr[j]
```

```
$   IF i == n-1 AND j < n-2 THEN
```

```
$     j = j+1
```

```
$     i = j+1
```

```
$   NEXT i
```